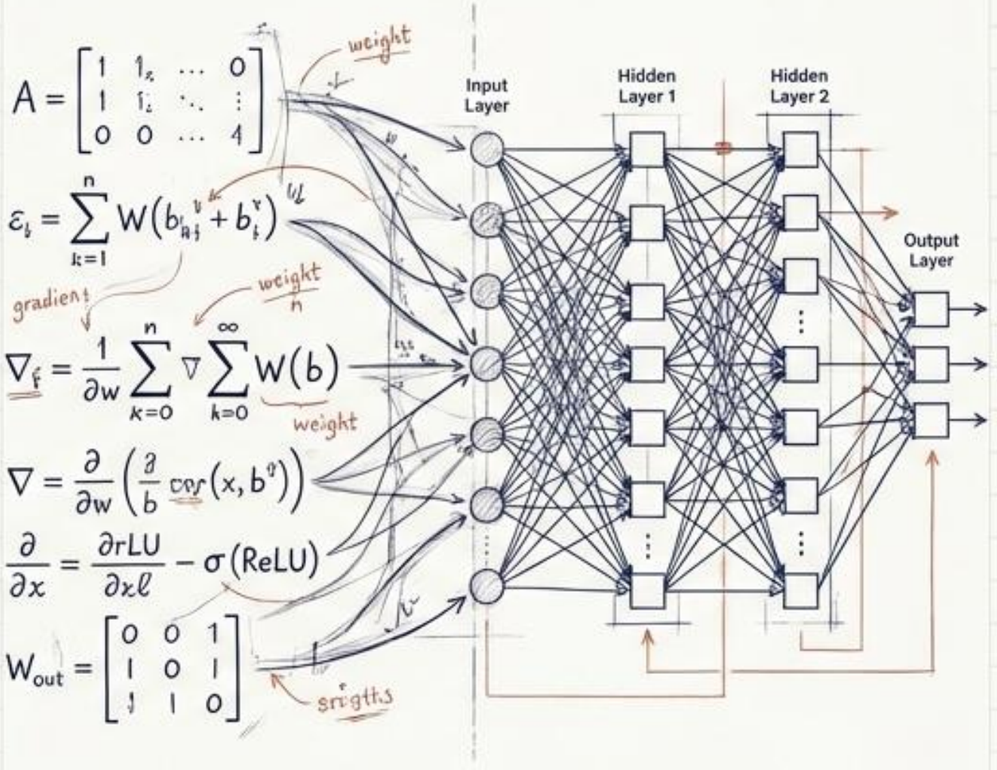
AIAA-3201 Week-06 Lab

Building and Inspecting Neural Networks

From Low-level Logic to High-level Vision

Objective: Demystify the 'black box' of Deep Learning by bridging low-level mathematical logic with high-level framework implementation.



NotebookLM

Environment Setup and Lab Deliverables

Environment Requirements

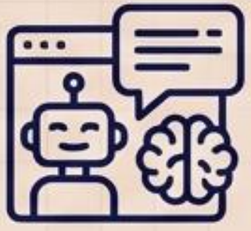


- Python 3.8+
- PyTorch
- numpy, matplotlib, scikit-learn

Submission Checklist



- `part1_numpy_backprop.py`
- `part2_pytorch_activations.py`
- `part3_cnn_dimensions.py`
- 1 PDF Report (Screenshots and short answers)



Engineering Pro-Tip: Deep learning environments can cause dependency conflicts. Do not hesitate to use LLMs to help debug installation errors or interpret Python tracebacks!

NotebookLM

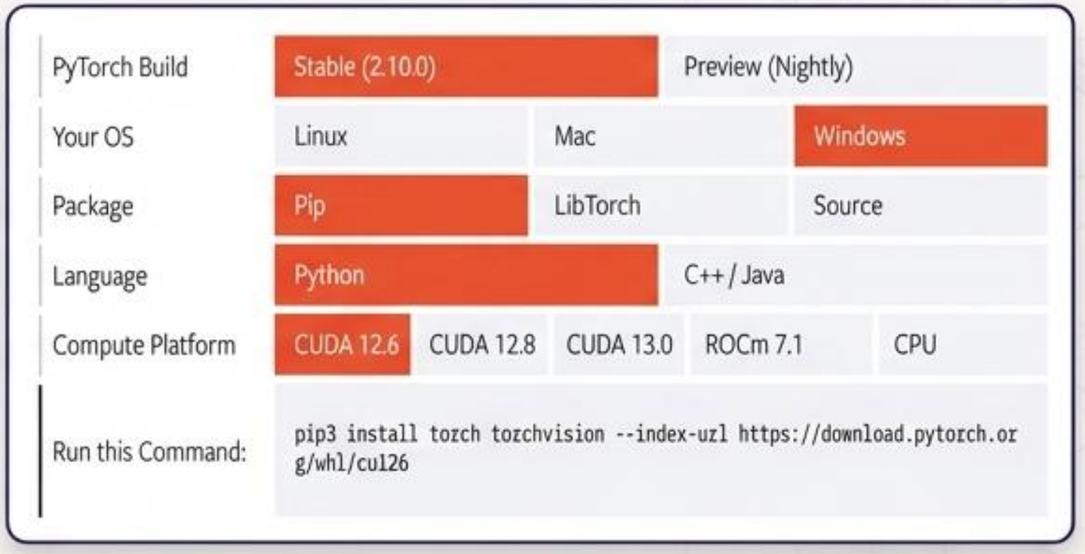
Configuring Your Local PyTorch Environment

The Official Guide: Generate your install command at pytorch.org/get-started/locally/.

Customized Matrix: Select your specific OS, Package Manager (Pip/Conda), and Compute Platform.

No GPU? No Problem: Select the CPU compute platform if you lack a dedicated GPU.

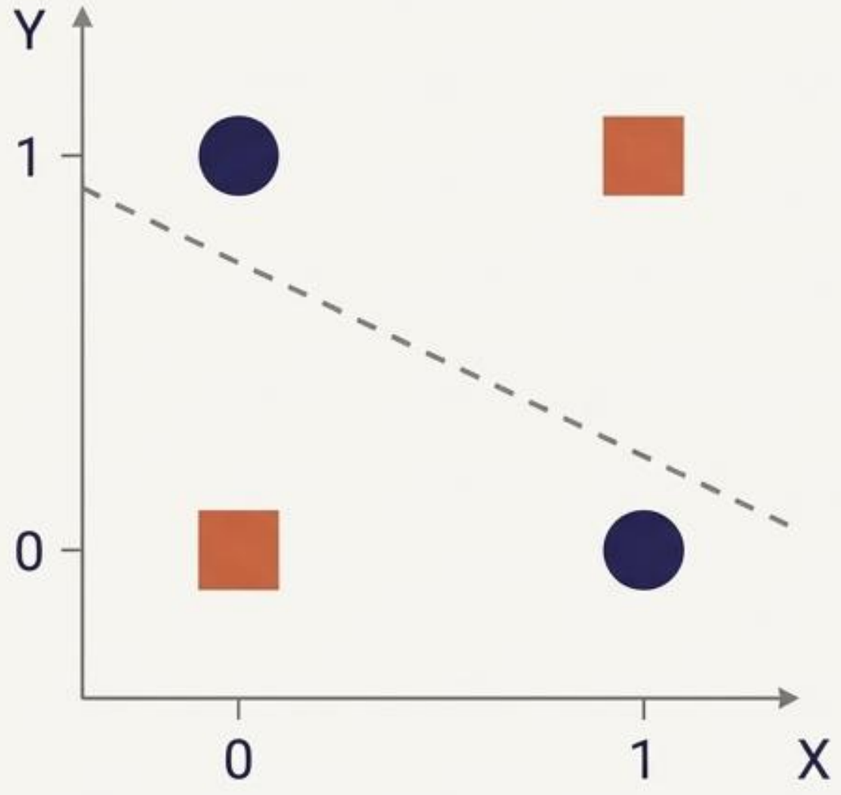
Pro-Tip for Stability: Clicking 'install previous versions of PyTorch' often provides a more heavily tested, stable environment than bleeding-edge releases.



Select your preferences on the PyTorch website to run the the correct install command.

NotebookLM

Defeating the XOR Problem from Scratch



The Problem: Purely linear classifiers cannot solve non-linear problems like XOR.

The Task (part1_numpy_backprop.py):
Implement the forward and backward passes of a 2-layer MLP using only raw NumPy. PyTorch is strictly forbidden here.

Fill in the TODOs:

- ☐ Implement the Sigmoid function.
- ☐ Forward matrix multiplications:
  $h = \sigma(XW_1 + b_1)$ and $y_pred = \sigma(hW_2 + b_2)$
- ☐ Error derivatives (chain rule):
  Start with $E = \frac{1}{2}(y_pred - Y)^2$. Calculate $\frac{\partial E}{\partial y_pred} = (y_pred - Y)$, multiply by the Sigmoid derivative, and backpropagate.
- ☐ Execute weight updates.

NotebookLM

Validating NumPy Logic and Exploring Convergence

Sanity Check: Run the code for 5 iterations. Verify that the loss successfully decreases.

Target Output: Your predictions should start moving toward [0, 1, 1, 0]. Add a screenshot of this table to your PDF Report.

```
================================================
Running Part 1: Numpy Backpropagation (XOR)
================================================
Iter  | Loss     | Output (y_pred)                  | Target       | W2 Mean
-------------------------------------------------------------------------------
1     | 0.1431   | [0.3345, 0.3180, 0.3121, 0.2886] | [0, 1, 1, 0] | -0.6762
2     | 0.1417   | [0.3419, 0.3176, 0.3198, 0.2976] | [0, 1, 1, 0] | -0.6847
3     | 0.1404   | [0.3491, 0.3251, 0.3273, 0.3053] | [0, 1, 1, 0] | -0.6526
4     | 0.1391   | [0.2562, 0.3325, 0.3347, 0.3129] | [0, 1, 1, 0] | -0.6428
5     | 0.1379   | [0.3631, 0.3397, 0.3419, 0.3203] | [0, 1, 1, 0] | -0.6324

========================================
FINAL MODEL PARAMETERS
========================================
W1 (Input -> Hidden):
[[-0.20041807  0.50384432]
 [ 0.45688416  0.18718248]]

b1 (Hidden Biases):
[[-0.81499736 -0.81457224]]

W2 (Hidden -> Output):
[[-0.63887343]
 [-0.62678652]]

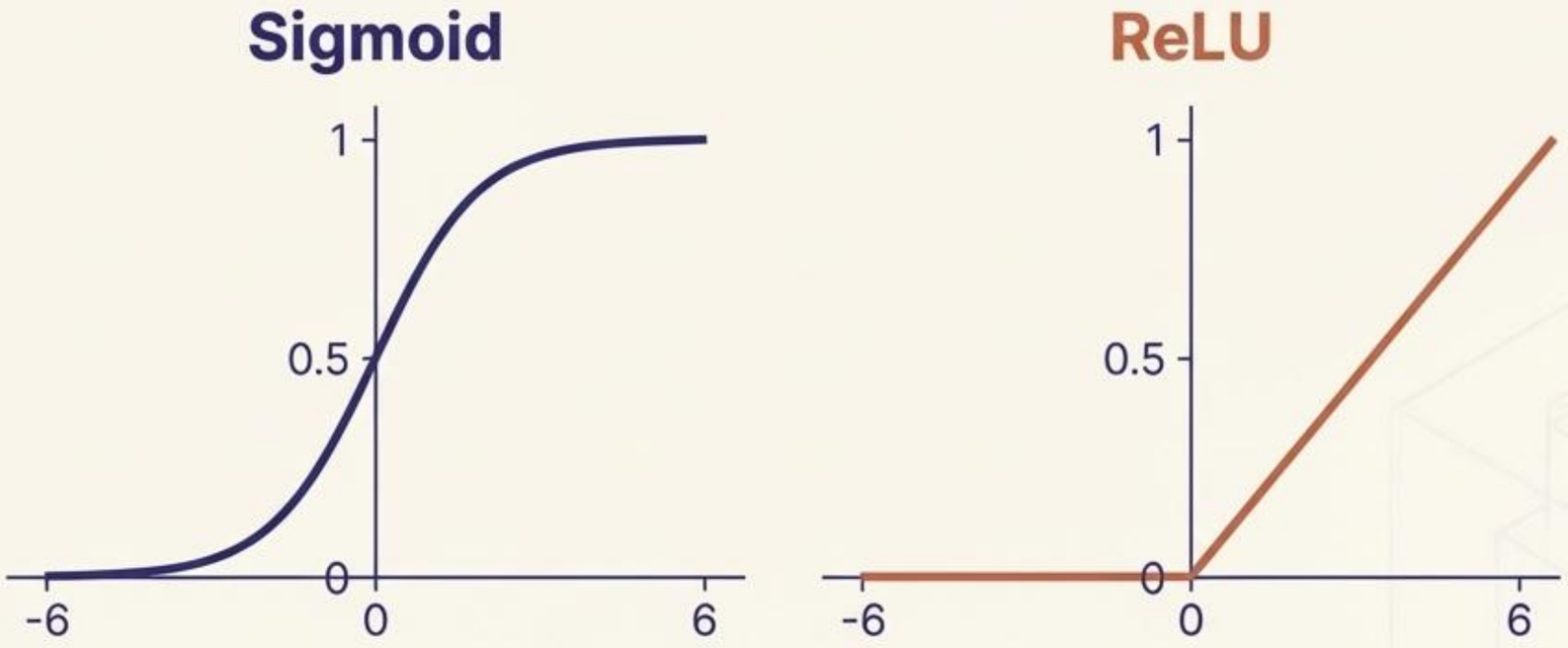
b2 (Output Biases):
[[0.69497252]]
========================================
```

Example terminal output showing iterations, loss, predictions, targets, and final model parameters.

Exploration Task: Increase iterations to 100, 5000, or 10000. If it gets stuck, ask why: Is it the learning rate? The hidden layer size? A bad random seed? Or the infamous vanishing gradient of the Sigmoid function?

NotebookLM

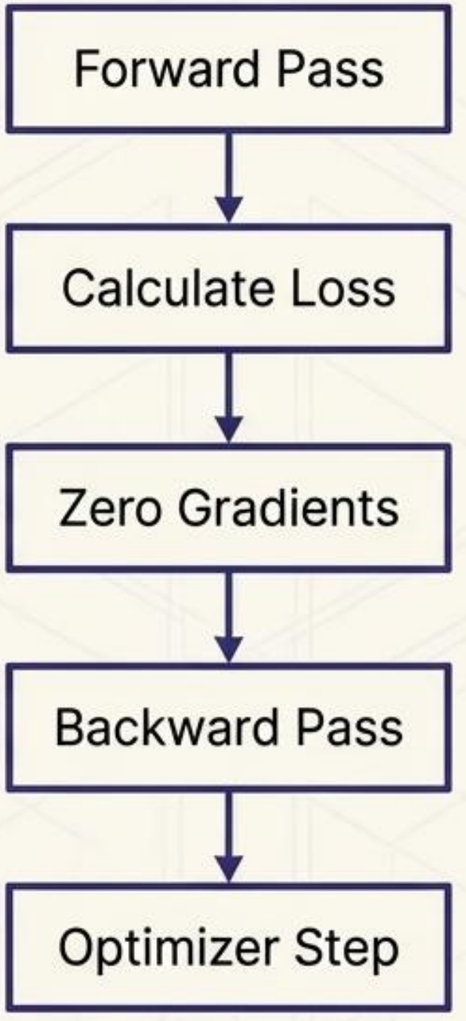
Sigmoid vs. ReLU in the Activation Arena



The Hypothesis: Are ReLU networks really faster and better than Sigmoid networks?

The Task (`part2_pytorch_activations.py`): Complete a 3-layer MLP in PyTorch to classify a non-linear dataset of concentric circles. Complete the __init__ and forward methods.

Master the PyTorch Loop:



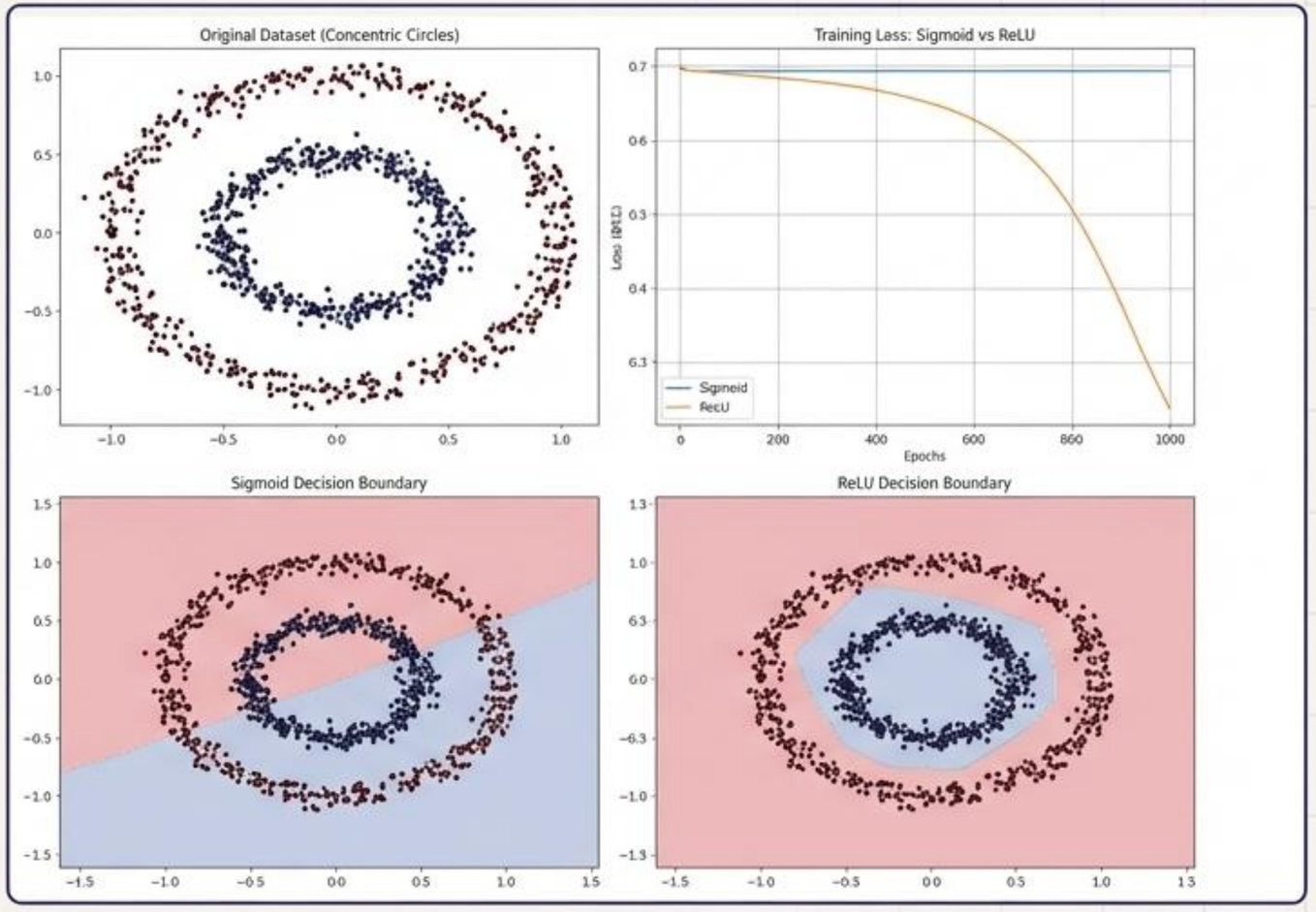
NotebookLM

Merriweather: Analyzing Activation Impacts on Model Convergence

Automated Verification: Your script will train both models and generate a comprehensive 4-panel plot mapping the battle.

Analyze the Results: Look closely at the decision boundaries and the loss curves. Notice how and when each network learns.

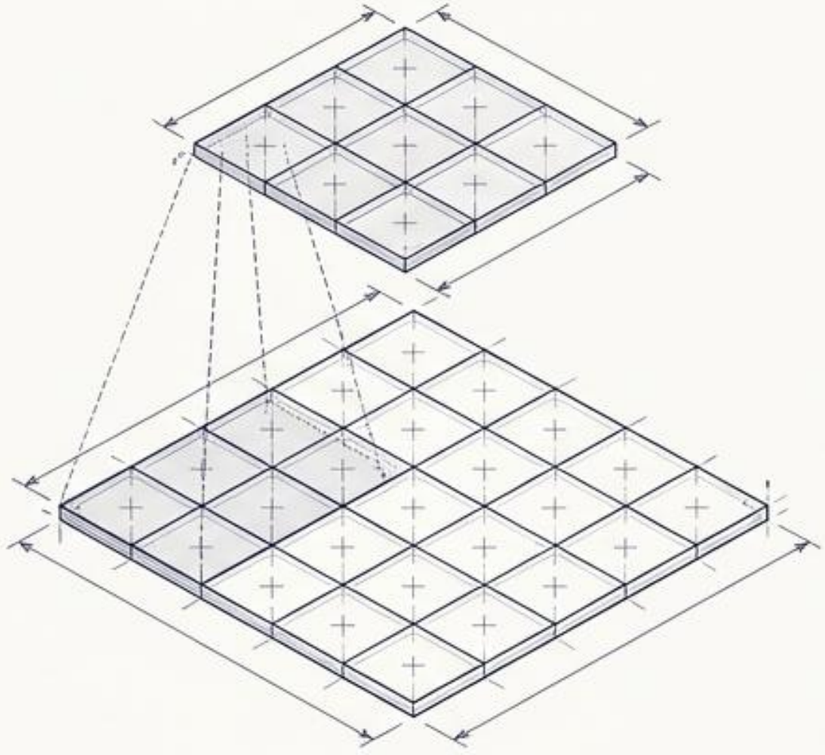
Report Deliverable: Save activation_comprehensive_analysis.png and write a short paragraph in your report explaining the difference in convergence (Hint: Think back to the vanishing gradients discussed in Part 1!).



Example output showing original data, loss curves, and decision boundaries for both Sigmoid and ReLU.

NotebookLM

Calculating Spatial Dimensions and Parameter Counts



Hand-Calculate First:

Output Dimensions:

```
(N + 2P - F) / stride + 1
```

Parameter Count:

```
C_out * (C_in * F * F + 1)
```

The Architect's Rule: Master spatial dimensions and parameter counts before building massive CNNs.

The Task (part3_cnn_dimensions.py): Run the script to generate randomized layer settings. **STOP!** Do not write the PyTorch code yet.

NotebookLM

Using PyTorch as a Verification Tool



Trust, but Verify: Now that you have hand-calculated the architectural dimensions, fill in the TODO to instantiate **nn.Conv2d** with those exact parameters.

Run the Verification: Execute the script again. Let PyTorch's internal dimension tracking verify if your human math was correct.

Report Deliverable: Take a screenshot of the terminal output verifying your hand-calculated CNN dimensions and parameters. You have successfully unboxed the deep learning pipeline!

```
============================================================
Running Part 3: CNN Dimension & Parameter Calculator
============================================================

============================================================
CNN Dimension & Parameter Calculator (Verification Tool)
============================================================

Given the following input and convolutional layer settings:
  - Input Image Size:  224x224
  - Input Channels:    3
  - Output Channels:   64
  - Filter Size (FxF): 5x5
  - Stride:            1
  - Padding:           1
============================================================

STOP! Before looking at the output below, hand-calculate:
1. The spatial dimensions of the output feature map. Formula: (N + 2P - F) / stride + 1
2. The total number of parameters in this layer (including bias).

Once calculated, run this script to verify your answers.

[VERIFICATION RESULTS]
-> Expected Output Shape: 222x222x64 (HxWxC_out)
-> PyTorch Tensor Shape: [1, 64, 222, 222] (Batch, Channel, Height, Width)
-> Total Layer Parameters: 4864
============================================================
```

NotebookLM